Open in Colab

Prelab 10.3 Machine Learning 3 - Real-time Sound Recognition for Classification

Load CNN model on Raspberry Pi

3.1 librosa installation on Raspberry Pi

In Lab 10, we will implement sound recognition on Raspberry Pi using the CNN model in the previous section. The librosa package will be used for the feature extraction of the CNN model. Follow the steps below to install librosa on your Raspberry Pi. If you have not installed TensorFlow on Raspberry Pi yet, please go to Prelab 9.1 and install it.

1. To install librosa, run the following command on the Terminal of your Raspberry Pi:

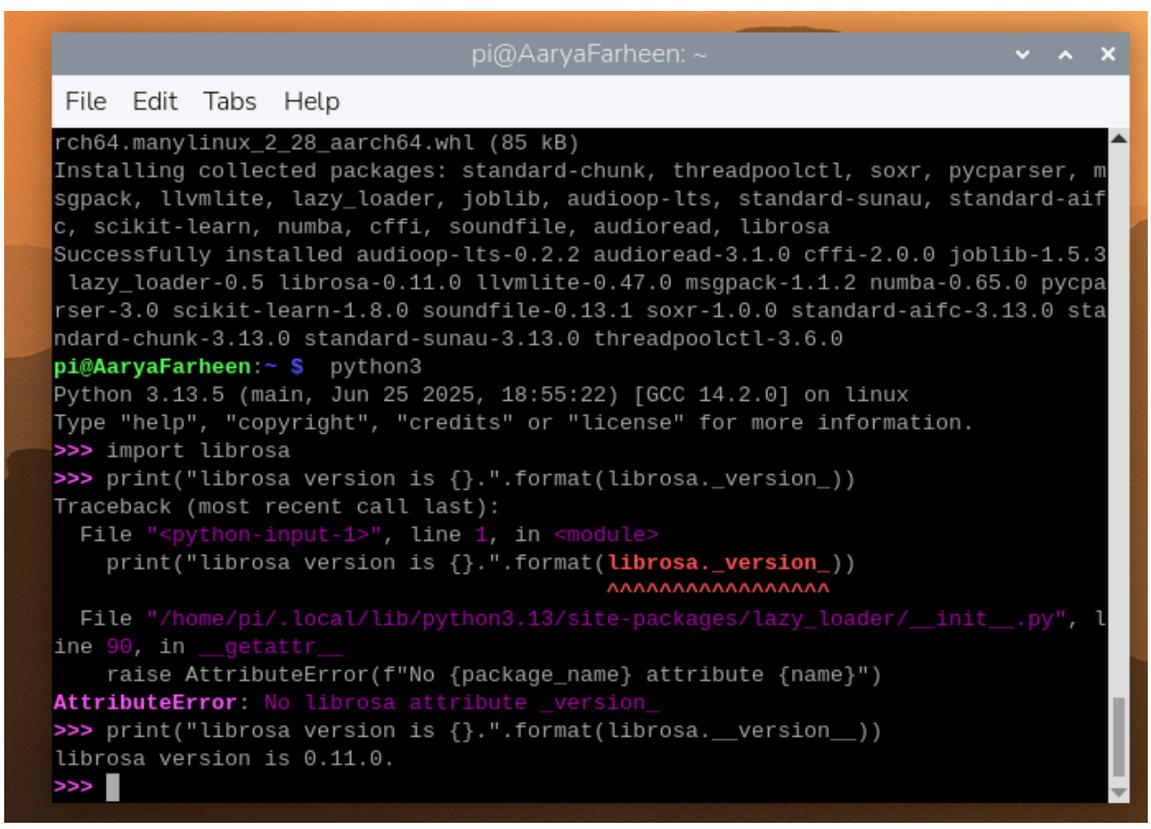
Raspberry Pi - Terminal

```
pip3 install librosa
```

Now your are ready to use librosa on Raspberry pi. Perform Task 1 to see if librosa is installed successfully on your Raspberry Pi.

Task 3.1

By running the example Python script below, load librosa and check the version as Figure 12 below and then attach it to the report.



Raspberry Pi - Terminal & Python

```
$ python3
>>> import librosa
>>> print("librosa version is {}.".format(librosa.__version__))
```

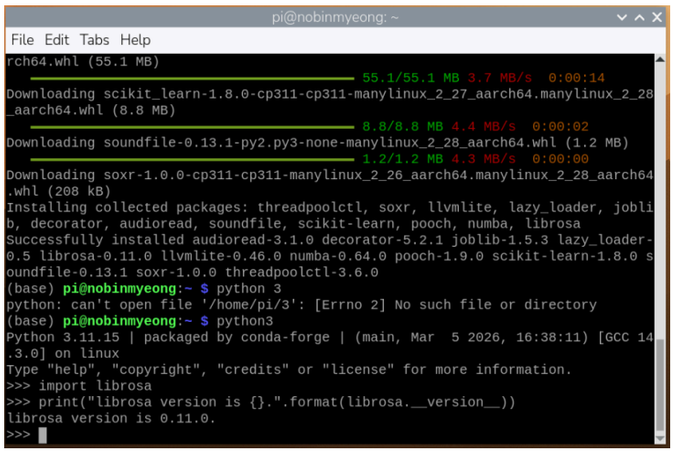
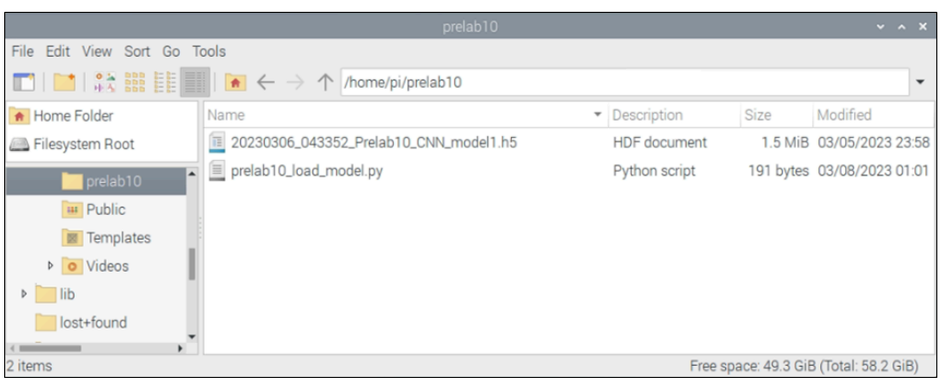


Figure 12 librosa version check (0.11.1)

3.2 Load CNN model on Raspberry Pi

Let's load and test the CNN model which you finally developed on Raspberry Pi. Please follow the steps below.

1. Create a working directory which your model and Python file will be in.
   a. The directory example (full path): '/home/pi/prelab10'
2. Copy and paste your final model binary file (*.h5 format) in the previous section to the directory ('/home/pi/prelab10' in this example).
   a. In this example, the model filename is '*20230306_043352_Prelab10_CNN_model1.h5*'.
3. Create a Python file as 'prelab10_load_model.py' on the same directory.



4. Code the Python script below in the created Python file ('prelab10_load_model.py') and then save it.

Python - Python3 (prelab10_load_model.py)

```
import tensorflow as tf

model = tf.keras.models.load_model('20230306_043352_Prelab10_CNN_model1.h5',  compile=False) # load your

print(model.summary()) # print out model (CNN) summary
```

Now, you are ready to use the model you created in Lab 10. Perform Task 3.2 below to see if the model is loaded successfully on your Raspberry Pi.

Task 3.2

1. After performing the stpes above, run the Python file. Is the result as you expected?
2. Capture the result of Terminal or Thonny Shell as Figure 13 and attach it to the report.

Yes, the results are very much expected.

```
pi@AaryaFarheen: ~/prelab10
File  Edit  Tabs  Help
(base) pi@AaryaFarheen:~/prelab10 $ python3 prelab10_load_model.py
Model: "sequential"
```

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 128, 93, 1)	0
conv2d (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_1 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten (Flatten)	(None, 1872)	0
dense (Dense)	(None, 64)	119,872
dense_1 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

```
Total params: 129,827 (507.14 KB)
Trainable params: 129,827 (507.14 KB)
Non-trainable params: 0 (0.00 B)
(base) pi@AaryaFarheen:~/prelab10 $
```

Figure 13 Check the model summary

3.3 Preparing implementation on Raspberry Pi

In Lab 10, to implement the CNN model on Raspberry Pi, **TensorFlow Lite** will be used. The TensorFlow Lite is a special feature and mainly designed for embedded and edge devices such as mobile, MCU, edge computer, and so on. It uses a custom memory allocator for execution latency and minimum load. Please go to TensorFlow Lite and look over it. We will use TensorFlow Lite Interpreter API.

As in the data collection via MTConnect sound stream for the training dataset, we will use the same sound sensor data flow for real-time implementation. By performing Task 3.3, come up with your monitoring system, logic, data flow, and so on so that you can realize the sound recognition of the operational state of the vacuum pump for real-time web-based monitoring.

Task 3.3

Considering the requirement below, describe the data flow pipeline (system configuration) and software you need to use.

- Sensor: Internal sound sensor (Stethoscope + microphone)
- Target machine: Vacuum pump
- Middleware 1 (server side, for sound stream): MTConnect
- Middleware 2 (Raspberry Pi side, for taking monitoring items): MTConnect
- Database: MySQL
- Visualization: Grafana
- Monitoring target(s):
    - Operational state of the machine (Event, OFF/ON-airleaking/ON-vacuum)
    - Sound level of the machine (Sample, dB unit)

Data Flow Pipeline / System Configuration

The monitoring system is designed to collect sound from a **vacuum pump** using an **internal sound sensor** made of a **stethoscope and microphone**, stream the sound data through **MTConnect**, process the required monitoring items on a **Raspberry Pi**, store the results in a **MySQL database**, and finally visualize them in **Grafana**.

System Configuration

The overall data flow pipeline is shown below:

```
[Vacuum Pump]
      ↓
[Internal Sound Sensor]
(Stethoscope + Microphone)
      ↓
[Sound Acquisition Device / Edge Input]
      ↓
[MTConnect Adapter / Agent on Server Side]
  - Streams raw or chunked sound data
      ↓
[Network Communication]
      ↓
[Raspberry Pi]
  - MTConnect client / subscriber
  - Reads monitoring items from MTConnect
  - Signal processing
  - Machine state classification
  - Sound level calculation
      ↓
[MySQL Database]
  - Stores timestamp
  - Operational state
  - Sound level (dB)
      ↓
[Grafana Dashboard]
  - Real-time and historical visualization
```

Data Flow Pipeline (Short Description)

🔁 Data Flow

Sound from the **vacuum pump** is captured using a **stethoscope + microphone sensor**.
This signal is streamed through **MTConnect (server side)**.

The **Raspberry Pi** acts as a client using **MTConnect**, where it:

- Processes the sound signal
- Classifies machine state (**OFF / ON-airleaking / ON-vacuum**)
- Computes sound level (**dB**)

The processed data is stored in a **MySQL database**, and **Grafana** is used to visualize:

- Machine state (event)
- Sound level (time-series)

🧰 Software Used

- **MTConnect Adapter & Agent** – for sound data streaming
- **Python (Raspberry Pi)** – data processing and classification
- **Audio Libraries** – `numpy`, `scipy`, `librosa/torchaudio`
- **MySQL** – data storage
- **Grafana** – dashboard visualization

Task 3.4

Using the MTConnect sound stream information given below, create a Python code to take current 1 second data to ingest them to the CNN model.

- Server domain: mepotrb16.ecn.purdue.edu
- MTConnect port: 5001
- Device.xml (probe) info is below.
- Measurement format: 16 Int (signed)

```
<?xml version="1.0" encoding="UTF-8"?>
<MTConnectDevices xmlns:m="urn:mtconnect.org:MTConnectDevices:1.1" xmlns:xsi="http://www.w3.org/2001/XML
    <Header creationTime="2010-03-04T18:44:40+00:00" sender="localhost" instanceId="1267728234" bufferSi
    <Devices>
        <Device id="sound1" name="sound1" uuid="sound1">
            <Description>ME597 Seound sensor at vacuum pump</Description>
            <DataItem category="SAMPLE" id="sensor1" name="sensor1" representation="TIME_SERIES" type="D
        </Device>
    </Devices>
</MTConnectDevices>
```

- A current response is below.

```
<?xml version="1.0" encoding="UTF-8"?>
<MTConnectStreams xmlns:m="urn:mtconnect.org:MTConnectStreams:1.4" xmlns="urn:mtconnect.org:MTConnectStr
  <Header creationTime="2023-03-08T06:48:37Z" sender="lamm-UltraPoint" instanceId="1676323553" version="
  <Streams>
    <DeviceStream name="sound1" uuid="sound1">
      <ComponentStream component="Device" name="sound1" componentId="sound1">
        <Samples>
          <DisplacementTimeSeries dataItemId="sensor1" timestamp="2023-03-08T06:48:37.274686Z" name="sen
        </Samples>
        <Events>
          <AssetChanged dataItemId="sound1_asset_chg" timestamp="2023-02-13T21:25:53.789634Z" sequence="
          <AssetRemoved dataItemId="sound1_asset_rem" timestamp="2023-02-13T21:25:53.789634Z" sequence="
          <Availability dataItemId="sound1_avail" timestamp="2023-02-13T21:45:55.179863Z" sequence="2737
        </Events>
      </ComponentStream>
    </DeviceStream>
  </Streams>
</MTConnectStreams>
```

MTConnect 1-Second Data Ingestion for CNN

🔁 Description

The MTConnect server streams sound data as **DisplacementTimeSeries** with:

- Sample rate = **48000 Hz**
- Each sequence = **2048 samples (Int16)** :contentReference[oaicite:0]{index=0}

To get ** 1 second data**, we collect 23 sequences: 2048 x 23 ≈ 47104 samples (~1 sec)

```
### If you code a Python script for Task 3.4, please copy and paste your code here.

import requests
import numpy as np
import tensorflow as tf
import librosa
import xml.etree.ElementTree as ET

# ============================
```

```
# CONFIG
# ==============================
MODEL_PATH = "20260407_182820_Prelab10_CNN_model1.h5"
MTCONNECT_URL = "http://mepotrb16.ecn.purdue.edu:5001/current"

SAMPLE_RATE = 48000
TARGET_SHAPE = (128, 93)   # your mel spectrogram

# ==============================
# LOAD MODEL
# ==============================
model = tf.keras.models.load_model(MODEL_PATH, compile=False)
print("✅ Model loaded")

# ==============================
# PARSE MTCONNECT XML
# ==============================
def get_audio_chunk():
    response = requests.get(MTCONNECT_URL)
    root = ET.fromstring(response.content)

    # Find DisplacementTimeSeries
    ns = {'m': 'urn:mtconnect.org:MTConnectStreams:1.4'}
    data = root.find('.//m:DisplacementTimeSeries', ns)

    if data is None:
        return None

    values = data.text.strip().split()
    values = np.array(values, dtype=np.int16)

    return values

# ==============================
# CREATE 1 SECOND WINDOW
# ==============================
buffer = []

def get_1sec_audio():
    global buffer

    while len(buffer) < SAMPLE_RATE:
        chunk = get_audio_chunk()
        if chunk is None:
            continue
        buffer.extend(chunk)

    # Take 1 second
    audio = np.array(buffer[:SAMPLE_RATE], dtype=np.float32)
    buffer = buffer[SAMPLE_RATE:]  # slide window

    # Normalize (important)
    audio = audio / 32768.0

    return audio

# ==============================
# MEL-SPECTROGRAM
# ==============================
def preprocess(audio):
    mel = librosa.feature.melspectrogram(
        y=audio,
        sr=SAMPLE_RATE,
        n_mels=128
    )

    mel_db = librosa.power_to_db(mel, ref=np.max)

    # Resize to match training shape
    mel_db = librosa.util.fix_length(mel_db, size=93, axis=1)

    # Add channel + batch
    mel_db = mel_db[..., np.newaxis]
    mel_db = np.expand_dims(mel_db, axis=0)

    return mel_db
```

```
# ==============================
# PREDICTION LOOP
# ==============================
while True:
    audio = get_1sec_audio()

    x = preprocess(audio)

    pred = model.predict(x)
    label = np.argmax(pred)

    print(f"Prediction: {label} | Probabilities: {pred}")
```

3.4 (Optional) Further reading

The list below is recent related research using the internal sound sensor and sound monitoring/recognition published in academic journals. If you are interested in detail, please go through them.

- Yun, H., Kim, H., Kim, E. and Jun, M.B., 2020. Development of internal sound sensor using stethoscope and its applications for machine monitoring. Procedia Manufacturing, 48, pp.1072-1078. (https://doi.org/10.1016/j.promfg.2020.05.147)
- Kim, E., Yun, H., Jun, M.B.G., Kim, K. and Cha, S.W., 2021, June. Multiple Sound Sensors And Fusion In Modern CNN-Based Machine State Prediction. In International Manufacturing Science and Engineering Conference (Vol. 85062, p. V001T05A017). American Society of Mechanical Engineers. (https://doi.org/10.1115/MSEC2021-64036)
- Yun, H., Kim, H., Jeong, Y.H. and Jun, M.B., 2023. Autoencoder-based anomaly detection of industrial robot arm using stethoscope based internal sound sensor. Journal of Intelligent Manufacturing, 34(3), pp.1427-1444. (https://doi.org/10.1007/s10845-021-01862-4)
- Kim, E., Yun, H., Araujo, O.C., and Jun, M.B.G., 2023, Sound Recognition based on Convolutional Neural Network for Real-Time Cutting State Monitoring of Tube Cutting Machine. International Journal of Precision Engineering and Manufacturing-Smart Technology, Vol. 1, No. 1, pp.1-18. (https://doi.org/10.57062/ijpem-st.2022.0038)

Get back to Lab Index Page